RNN Intuition

Structured Notes

Sequential Data, Need for Memory,

Hidden States,
RNN Update Equation, Task Types, and Limitations

Contents

1 Why CNN Is Not Enough **2**
1.1 Key Differences: Image Data vs Sequential Data . . . 2
1.2 Why CNN Struggles with Sequential Data . . . 2

2 Examples of Sequential Data **2**
2.1 Text Data . . . 2
2.2 Speech Data . . . 3
2.3 Time Series Data . . . 3

3 Sequence Tasks **3**
3.1 Sequence Classification (Many-to-One) . . . 3
3.2 Sequence Prediction (Many-to-Many) . . . 3
3.3 Sequence Generation (Many-to-Many) . . . 3
3.4 Summary of Sequence Tasks . . . 4

4 Vocabulary and Tokens **4**

5 Need for Memory in Neural Networks **4**
5.1 Why Memory is Important . . . 5
5.2 Traditional NN vs Sequential Processing . . . 5
5.3 Variable Length Challenges . . . 5

6 The Idea of Recurrence **5**

7 Hidden State: The RNN Update Equation **6**
7.1 Why Use tanh Activation? . . . 6
7.2 Connection to ARIMA . . . 7

8 RNN Task Types and Use Cases **7**
8.1 Many-to-One . . . 7
8.2 One-to-Many . . . 7
8.3 Many-to-Many . . . 7
8.4 RNN Use Cases by Domain . . . 8

9 Limitations of Simple RNN **8**
9.1 Long-Term Dependency Problem . . . 8
9.2 Vanishing Gradient Problem . . . 8
9.3 Exploding Gradient Problem . . . 8
9.4 Practical Consequences . . . 9

10 Motivation for LSTM and GRU **9**
10.1 Transformers: A Brief Preview . . . 9

11 Summary **9**

1 Why CNN Is Not Enough

CNN has been highly effective for image-based tasks. Images have a very specific structure: fixed size, spatial relationships, no time dependency, and all information is available at once. However, not all data looks like images.

Sequential Data

Sequential data is data where **order matters**. Each data point depends on previous data points, and changing the order changes the meaning. Examples include text, speech, time series, video, and sensor data.

For example, the sentence "I love deep learning" is meaningful. If the order is changed to "learning deep love I," the meaning is lost completely. Every word depends on the past words and every word carries meaning in context.

1.1 Key Differences: Image Data vs Sequential Data

Aspect	Image Data	Sequential Data
Structure	Spatial structure	Temporal structure
Size	Fixed size	Variable length
Ordering	Order does not matter	Order matters
Dependency	No time dependency	Time dependency present
Information availability	All information available at once	Information arrives in sequence

1.2 Why CNN Struggles with Sequential Data

CNN struggles with sequential data because:

- CNN assumes **fixed input size** — but sequences have variable length
- CNN has **no concept of memory** — nothing is saved across inputs
- CNN performs only **local pattern detection** — no global context understanding
- Sequences require **memory of past information** and **context understanding**

Key Insight

In most real-world scenarios, companies work with order-dependent data. Text, speech, time series, sensor data — all require sequential processing. CNN is suited only for computer vision applications (image classification, face recognition, etc.), and even those are increasingly being replaced by transformers.

This leads to a fundamental question: **How do we design neural networks that understand order and memory?** This is where RNN comes into the picture.

2 Examples of Sequential Data

2.1 Text Data

Each word depends on the previous word. "I am learning" is meaningful, but "learning am I" changes the meaning. Order changes meaning, so sequential processing is required.

2.2 Speech Data

Speech is a time-dependent signal. Sound at time t depends on sound at time $t-1$ and $t-2$. Speech recognition requires understanding the sequence of sounds in order.

2.3 Time Series Data

Stock prices, weather data, sensor data, ECG signals — in all these cases, future values depend on past values. Temperature data, for instance, requires knowledge of previous days to predict the next day.

RNN as a Foundation

RNN builds a base for NLP (Natural Language Processing). RNN is the starting point that leads to LSTM, GRU, and eventually Transformers. From an industry and interview perspective, understanding RNN is essential even though Transformers have largely replaced RNN in production applications.

3 Sequence Tasks

There are three major types of sequence tasks:

3.1 Sequence Classification (Many-to-One)

Sequence Classification

Input is a full sequence, output is a single label. The model reads the entire sequence and makes one final decision.

Examples:

- **Sentiment analysis:** "I love this movie" → Positive sentiment
- **ECG signal classification:** Time series ECG signal → Normal / Abnormal
- **Stock trading decision:** Previous stock prices → Buy / Sell

3.2 Sequence Prediction (Many-to-Many)

Sequence Prediction

Input is a past sequence, output is a future sequence. Past values are used to predict next values.

Examples:

- Stock price prediction: past values → predict future values
- Weather forecasting: past temperature data → predict future temperatures

3.3 Sequence Generation (Many-to-Many)

Sequence Generation

A start token or input sequence is provided, and the model generates a complete output sequence.

Examples:

- **Text generation:** Input "deep learning is" → Output "deep learning is transforming artificial intelligence"
- **Language translation:** Input in English "I love AI" → Output in French (translated sentence)

3.4 Summary of Sequence Tasks

Task Type	Mapping	Input	Output
Classification	Many-to-One	Full sequence	Single label
Prediction	Many-to-Many	Past sequence	Future values
Generation	Many-to-Many	Start token/sequence	Generated sequence

4 Vocabulary and Tokens

Neural networks cannot understand text directly. Neural networks only understand numbers (vectors, matrices, and tensors). Therefore, text must be converted into numerical representation. This introduces two important concepts: **vocabulary** and **tokens**.

Vocabulary

Vocabulary is the set of all unique words in the dataset.

Example: Given the dataset with sentences "I love AI" and "I love deep learning":

- Vocabulary = {I, love, AI, deep, learning}
- Vocabulary size = 5

Tokens

Tokens are the numeric indices assigned to each word in the vocabulary.

Example:

- I $\rightarrow$ 0
- love $\rightarrow$ 1
- AI $\rightarrow$ 2
- deep $\rightarrow$ 3
- learning $\rightarrow$ 4

This process of converting words to numeric indices is called **tokenization**.

Why do we need tokens? Neural networks work with vectors, matrices, and tensors. Text must be converted into numeric form before it can be processed.

Tokenization process:

1. **Step 1 — Tokenization:** Break the sentence into individual words. "I love deep learning" $\rightarrow$ [I, love, deep, learning]
2. **Step 2 — Convert to numbers:** Map each word to its index. [I, love, deep, learning] $\rightarrow$ [0, 1, 3, 4]

Looking Ahead

Tokenization is a fundamental concept for LLMs (Large Language Models). In future study, these numeric tokens are converted into vector embeddings (meaningful vector representations) rather than simple integer indices. This will be covered in NLP topics including bag of words, word-to-vector, and other embedding techniques.

5 Need for Memory in Neural Networks

Three major challenges have been identified for processing sequential data:

1. **Order matters** — changing the order changes the meaning
2. **Variable length** — sequences can be of different lengths
3. **Long-term dependency** — the model must remember information from far back in the sequence

The most important requirement is that the neural network **must remember past information**.

5.1 Why Memory is Important

Consider the sentence: "I grew up in France, I speak fluent _____." To predict the blank (French), the model must remember "France," which appeared far earlier in the sequence. This is called **long-term dependency**.

Another example: Temperature data for Monday, Tuesday, Wednesday — to predict Thursday, the model must remember all three previous days.

5.2 Traditional NN vs Sequential Processing

Traditional Neural Network Processing

In a standard feed-forward neural network, each input is processed **independently**. There is no connection between successive inputs.

Example: Processing "I love deep learning" — the NN sees "I," then "love," then "deep," then "learning" — with no connection between them.

Sequential Processing (RNN Approach)

Instead of processing each input independently, sequential processing uses: **Input + Previous Memory → Output**.

- "I" → stored in memory
- "love" comes → connected with "I" in memory, memory updated
- "deep" comes → connected with previous memory, memory updated again
- "learning" → final prediction made using accumulated memory

In this manner, the model remembers the sequence.

5.3 Variable Length Challenges

Sequence Length

The **sequence length** is the number of words (or data points) in the input stream. This length is variable, which is a challenge because neural network input shapes are typically fixed.

Two techniques address variable length:

- **Padding:** Addition of zeros to make sequences of equal length
- **Truncation:** Cutting long sequences to a fixed size

6 The Idea of Recurrence

In a standard feed-forward neural network, each input is processed independently. This works well when inputs do not depend on each other. But in sequential data, the current input often depends on what came before — and sometimes on what came **long before**.

- In language, the meaning of a word depends on previous words
- In speech, the current sound depends on earlier sounds
- In time series, the present value is often related to past values

Instead of processing each input independently, the goal is to have a model that carries forward some information from the past.

Efficiency Constraint

Carrying forward **all** past information would result in exponential memory growth, which is too expensive. Therefore, only **some** information is carried forward at each time step. This selective memory is precisely the limitation that Transformers later addressed with the attention mechanism.

The core idea: at each time step, the model remembers previous information, receives the current input, then updates the memory. This loop is repeated at every time step — which is why it is **recursive** in nature and called a **Recurrent Neural Network**.

Sequential Processing at Time Step t:

$$\text{Memory}_{t-1} + \text{Input}_t \longrightarrow \text{Memory}_t$$

Memory_t is then used for the next time step $t+1$. This process keeps repeating:

$$\text{Memory}_{t-1}, \text{Input}_t \rightarrow \text{Memory}_t \rightarrow \text{Memory}_t, \text{Input}_{t+1} \rightarrow \text{Memory}_{t+1} \rightarrow \cdots$$

This repetitive processing through time is why RNN is called **recurrent**. The structure is analogous to a recursive function.

7 Hidden State: The RNN Update Equation

The **hidden state** acts like the network's memory. At time step t:

1. The network receives the current input x_t
2. It also uses the previous hidden state h_{t-1}
3. It computes the new hidden state h_t

Basic RNN Update Equation:

$$h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$$

Where:

- x_t — current input at time step t
- h_{t-1} — previous hidden state (memory from the last time step)
- h_t — new hidden state (updated memory)
- W_x — weight matrix for the input
- W_h — weight matrix for the previous hidden state
- b — bias term

The expression inside the tanh bracket is a linear transformation. At its core, everything is linear algebra. The same function is called for h_t, then h_{t+1}, and so on — the same function is applied again and again, which is why the network is recursive/recurrent.

7.1 Why Use tanh Activation?

Why $\tanh$ and Not ReLU or Sigmoid

tanh is chosen for the RNN update equation for the following reasons:

1. **Zero-centered output:** tanh outputs values in $[-1, 1]$ with zero at the center. This provides a stable transformation.
2. **Non-linearity:** Without an activation function, the model remains too limited (purely linear). tanh introduces the necessary non-linearity for learning complex relationships.
3. **Bounded output:** Values are kept between -1 and 1, preventing hidden state values from getting too large during repeated updates.
4. **Supports negative values:** Unlike ReLU (which outputs 0 for negative inputs), tanh can represent negative sentiment or negative relationships.

Why not ReLU?

- ReLU outputs range from 0 to infinity — the maximum can grow unbounded, causing exponential computation
- ReLU does not produce negative values, making it difficult to classify negative sentiments or model opposing relationships
- ReLU is not zero-centered (range is $[0, \infty)$)

7.2 Connection to ARIMA

The RNN hidden state concept connects directly to time series models like ARIMA and SARIMA. ARIMA captures time dependency in statistical models; RNN captures the same time dependency using neural networks with a recursive function. The core principle — using past information to predict future values — is the same.

8 RNN Task Types and Use Cases

RNN is not just a model for text — it is a **general-purpose framework for ordered data** where the past influences the present. It can be used with images, text, audio, or any data where order matters.

8.1 Many-to-One

A full sequence is given as input and one output is produced. The model reads the whole sequence and makes one final decision.

Use cases:

- Sentiment analysis (movie review $\rightarrow$ positive/negative)
- Spam detection
- Activity classification from sensor signals
- Disease classification from ECG and medical time series

8.2 One-to-Many

One input is given and a sequence is generated as output.

Use cases:

- Image captioning (image $\rightarrow$ descriptive caption)
- Music generation
- Text generation from a starting signal

8.3 Many-to-Many

A sequence is mapped to another sequence.

Use cases:

- Language translation (English sentence → Hindi sentence)
- Speech recognition
- Subtitle generation
- Sequence labeling

8.4 RNN Use Cases by Domain

Domain	Applications
Language	Next word prediction, sentiment analysis, translation, text generation
Speech	Speech-to-text, phoneme recognition, speaker modeling
Time Series	Stock price prediction, weather forecasting, machine sensor analysis, anomaly detection
Vision + Sequence	Image captioning (encoder-decoder), video understanding

Encoder-Decoder Architecture

When the task involves converting a single input (like an image) into an ordered sequence (like a caption), the **encoder-decoder** architecture is used. The encoder processes the input, and the decoder generates the sequence. This architecture is closely related to autoencoders and is fundamental to Transformers.

9 Limitations of Simple RNN

While RNN can process ordered data, carry memory through hidden states, and work on variable-length inputs, simple RNN has an important weakness: **it struggles when useful information must be remembered for a long time**.

9.1 Long-Term Dependency Problem

Consider the sentence: "The book that I bought in Paris last year was written in French, so I could finally understand the ending because I know _____."

To fill in the blank, the model needs to remember "Paris" and "French," which appear much earlier in the sequence. The most important clues are far from the prediction point.

9.2 Vanishing Gradient Problem

Vanishing Gradient

At each time step, the hidden state is updated repeatedly. During training, gradients must flow backward through many time steps (backpropagation through time). When this happens repeatedly:

- Gradients can become **very small** — this is the **vanishing gradient problem**
- Early time steps learn very slowly
- Distant past information is not preserved well
- Long-term dependencies are hard to learn

This leads to **exponential forgetting** of distant past information.

9.3 Exploding Gradient Problem

Exploding Gradient

Sometimes gradients can also become **very large** during backpropagation through time. This is called the **exploding gradient problem**, which causes numerical instability during training.

9.4 Practical Consequences

- Simple RNN works reasonably well for **short dependencies**
- It becomes weak when:
  - Sequences are very long
  - Important context appears far in the past
  - Memory must be preserved over many time steps

10 Motivation for LSTM and GRU

Because of the limitations of simple RNN, improved recurrent models were introduced:

- **LSTM** (Long Short-Term Memory)
- **GRU** (Gated Recurrent Unit)

These models are designed to **control memory more effectively** — they can decide **what to remember** and **what to forget**. The next natural question is: How do we design recurrent models that decide what to remember and what to forget?

Learning Path

The progression of topics follows a natural sequence: RNN (base) → LSTM and GRU (improved memory control) → NLP (applications) → Transformers (attention-based architecture that replaced sequential processing). Each builds upon the previous concept.

10.1 Transformers: A Brief Preview

Transformers addressed the fundamental limitation of RNN-based models. Instead of processing data word by word and carrying forward only partial information, Transformers use the **attention mechanism** to see the complete picture at once. The landmark paper "Attention Is All You Need" was the game changer that replaced sequential processing with parallel attention-based processing.

11 Summary

- **CNN is not enough** for sequential data because it lacks memory, assumes fixed input size, and performs only local pattern detection
- **Sequential data** includes text, speech, time series, video, and sensor data — all require order-dependent processing
- **Three sequence tasks:** classification (many-to-one), prediction (many-to-many), generation (many-to-many)
- **Vocabulary and tokens** are the bridge between text and neural networks — text must be converted to numeric form via tokenization
- **Memory is essential** — neural networks must remember past information for sequential processing
- **Hidden state** (h_t) acts as the network's memory, updated at each time step using the RNN update equation: $h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$
- tanh is used because it is zero-centered ($[-1, 1]$), provides non-linearity, and keeps hidden state values bounded
- **RNN is a general-purpose framework** for any ordered data, not limited to text
- **RNN task types:** many-to-one, one-to-many, many-to-many
- **Simple RNN limitations:** vanishing gradient problem and exploding gradient problem make long-term

dependencies hard to learn

- **LSTM and GRU** were developed to address these limitations by controlling what to remember and what to forget
- **Transformers** ultimately replaced RNN-based models by using attention to process entire sequences in parallel